TECHNICAL ENGINEERING & MODEL OPTIMIZATION REPORT

Natural-Prevalence F1 Optimization and Hard-Negative Suppression in QueryDiscussNet (QDN)

Project: Thoracic Pathology Multi-Label Classification (`vindr_project`) | **Environment:** `GPU-Server01` (`nih_venv`) | **Focus Classes:** Effusion, Pneumothorax, Cardiomegaly

1. Aim and Objectives

The primary objective of this engineering phase is to maximize the natural-prevalence F1 score and Area Under the Precision-Recall Curve (AUPRC) of the pretrained **QueryDiscussNet (QDN)** architecture (checkpoint `outputs/final/qdn/best_ema.pt`) across three high-priority thoracic pathologies: **Effusion**, **Pneumothorax**, and **Cardiomegaly**.

While the baseline QDN model demonstrates strong discriminative capability (AUROC), deployment under natural clinical prevalence exposes susceptibility to false positives (FPs) on confounding abnormal scans—specifically standard PA chest radiographs exhibiting co-existing multi-label conditions or chronic parenchymal scarring. This work establishes a rigorous post-processing and fine-tuning workflow to suppress confounder-induced false positives while preserving true-positive recall and strict patient-level split integrity.

2. End-to-End Optimization Workflow

Our optimization strategy progressed through four structured phases:

1. **Stage-1 Gate Override & Validation Threshold Calibration:** Leveraging the 12th output head (`Is_Abnormal`) as a Stage-1 abnormality gate alongside validation-tuned class-specific decision thresholds to establish our post-processed baseline metrics.
2. **False-Positive Mining & Metadata Forensics:** Extracting high-confidence false-positive predictions to inspect radiographic view positions, patient demographics, and co-existing disease labels.
3. **Root-Cause Diagnosis of NaN Losses & Architectural Mismatches:** Resolving structural discrepancies between standalone fine-tuning scripts and the native `QueryDiscussNet` codebase, cache directory paths, mixed-precision loss scaling, and Exponential Moving Average (EMA) warmup dynamics.
4. **Confounder-Aware Hard-Negative Fine-Tuning:** Implementing a native fine-tuning loop utilizing a float32 focal false-positive penalty (`HardNegativeSuppressionLoss`) and `WeightedRandomSampler` to suppress confounder activations directly inside the QDN query-discussion heads.

3. Phase 1: Stage-1 Abnormality Gate & Threshold Optimization

In the multi-label QDN setup, the model outputs 12 logits: the 11 target pathologies (`TARGET_11_CLASSES`) plus an overall abnormality indicator (`Is_Abnormal`) stored under `outputs/final/qdn/gate/`. To prevent false alarms on healthy ("safe house") scans, we applied a hard Stage-1 gate override ($\tau_{gate} = 0.30$): any scan whose predicted abnormality probability fell below 0.30 had its pathology probabilities zeroed out prior to thresholding.

To strictly prevent test-split data leakage, optimal decision thresholds for each focus pathology were swept exclusively on the natural-prevalence validation split (`val_probs.npy` and `val_labels.npy`) and subsequently evaluated on the held-out test split.

Focus Pathology	Stage-1 Gate (τ_{gate})	Evaluation Protocol	Post-Gate Test F1 (Baseline QDN)
Effusion	0.30	Validation-Tuned Threshold + Gate Override	**0.5312**
Pneumothorax	0.30	Validation-Tuned Threshold + Gate Override	**0.4695**
Cardiomegaly	0.30	Validation-Tuned Threshold + Gate Override	**0.4187**

4. Phase 2: False-Positive Forensics & Confounder Analysis

Following threshold calibration, we analyzed the remaining false positives across the test split, identifying 7,029 unique false-positive image instances (exported to `hard_negatives_to_mine.txt`). Cross-referencing these predictions against patient metadata revealed a critical insight:

- **Standard PA Dominance & Co-Existing Pathologies:** The highest-confidence false positives were not caused by portable AP projection artifacts alone, but predominantly occurred on standard Posteroanterior (PA) scans containing other legitimate abnormalities or chronic scarring (such as patient `00000211`).
- **Why Stage-1 Gating Cannot Filter Confounders:** Because these scans genuinely contain other thoracic findings, their ground-truth `Is_Abnormal` label is `1.0`, and the Stage-1 gate rightly assigns them a high abnormality score ($p_{gate} > 0.30$). Consequently, suppressing these false positives requires improving inter-class specificity inside the QDN model itself.

5. Phase 3: Technical Issues Encountered & Engineering Resolutions

During the transition from post-processing to model fine-tuning (`finetune_qdn.py`), several critical system and deep-learning engineering bottlenecks surfaced and were systematically resolved:

Issue / Error Encountered	Root Cause Analysis	Engineering Resolution
1. State-Dict Shape Mismatches & Uninitialized Layers `RuntimeError: size mismatch for proj4.weight...`	An ad-hoc wrapper class attempted to reconstruct QDN using standard `torchvision.models.resnet50` and 512-dim linear layers with `strict=False`, whereas the checkpoint used `timm`'s `resnet50d` (`embed_dim=256`), 3 shared `ReadDiscussStep` iterations, and 4 learned prototypes per class.	Replaced the ad-hoc wrapper by importing the exact native `QueryDiscussNet` class from `models.query_discuss_net` and loading `best_ema.pt` with `strict=True` (100% parameter match).
2. DataLoader Worker `FileNotFoundError` on PNGs	The standalone loader searched for raw images in `./images`, which did not exist in the workspace directory structure.	Located the pre-extracted 224×224 image cache directory at `/mnt/data/shyam/debarshi/vindr_project/cache_224` and connected it via the native `ChestXrayDataset` and `configs/top3_seed42.yaml`.
3. Catastrophic NaN Loss Explosion Around Batch 655	Computing BCE loss inside the FP16 `torch.amp.autocast` context caused logit overflow. Furthermore, skipping `loss.backward()` on `NaN` batches did not prevent corruption because `BatchNorm2d` running statistics had already updated during the forward pass.	Moved loss computation outside the `autocast` block and explicitly cast logits to 32-bit floating point (`logits.float()`) prior to BCE evaluation, matching `training/train_pipeline.py`.
4. Pretrained EMA Weight Destruction on Batch 1	`ModelEMA` initializes with `self.updates = 0` and uses a warmup schedule $d = (1 + u) / (10 + u)$. On step 1 ($d = 0.18$), 82% of the pretrained EMA checkpoint would be overwritten by noisy first-batch weights.	Explicitly locked `ema.updates = 5000` and `decay = 0.998` immediately after loading `best_ema.pt` to preserve pretrained representations.
5. Patient Split Disjointness vs. Test FP Lists	Because `load_saved_splits` enforces strict patient-level disjointness between train, val, and test splits, test-set filenames in `hard_negatives_to_mine.txt` never appear in `train.csv`.	Eliminated test-file string matching and engineered confounder mining directly on the training split using ground-truth multi-label indicators ($y_c = 0$ and $y_{abn} = 1$).

6. Phase 4: Native QDN Architecture & Hard-Negative Suppression Pipeline

The finalized fine-tuning script (`finetune_qdn.py`) integrates seamlessly with the repository's native modules:

6.1 Native QueryDiscussNet Forward Dynamics

As defined in `models/query_discuss_net.py`, the model extracts multi-scale feature maps (C_4 at stride 16 and C_5 at stride 32) from a `resnet50d` backbone, projects them to 256-dimensional patch tokens with 2D sine-cosine positional encodings, and iteratively refines 12 learned "doctor" queries across $T = 3$ shared steps of cross-attention (`read`), inter-disease self-attention (`talk`), and token refinement (`write`), augmented by 4 learned prototype vectors per disease.

6.2 Confounder-Focused Loss Function (`HardNegativeSuppressionLoss`)

To directly penalize false positives on the focus classes without degrading sensitivity, we formulated an asymmetric focal-weighted Binary Cross-Entropy loss evaluated in FP32. For each sample and class c, let $p_c = \sigma(z_c)$ be the detached predicted probability and $y_{abn} \in \{0, 1\}$ be the ground-truth abnormality flag. For each focus class $c \in \{Effusion, Pneumothorax, Cardiomegaly\}$, the sample weight w_c is defined as:

> **HARD-NEGATIVE SUPPRESSION WEIGHT FORMULATION**
>
> $$w_c = 1.0 + \mathbb{I}(y_c = 0) \cdot \alpha_{FP} \cdot (p_c)^{\gamma} \cdot (1.0 + 0.5 \cdot y_{abn})$$
>
> Where $\alpha_{FP} = 3.5$ is the base false-positive penalty factor and $\gamma = 2.0$ is the focal exponent. When a training image is negative for a focus pathology ($y_c = 0$) but the model predicts a high probability p_c—especially on abnormal confounder scans ($y_{abn} = 1$)—the loss penalty scales quadratically with prediction confidence, forcing the self-attention discussion step to disentangle co-existing diseases.

6.3 Balanced Confounder Sampling & Differential Learning Rates

- **Weighted Random Sampling:** Using `WeightedRandomSampler` on the 77,238 training scans, abnormal confounder scans ($y_{abn} = 1$ with no focus pathology) are sampled at `2.0x` weight, while positive focus-class scans are sampled at `2.5x` weight to maintain recall.
- **Differential AdamW Schedule:** The `resnet50d` backbone is fine-tuned at a conservative learning rate ($0.15 \times lr_{base}$) while the QDN query, discussion, and prototype heads are updated at $4.0\times$ the backbone rate over 2 epochs with cosine annealing and gradient norm clipping (`max_norm=2.0`).

7. Experimental Results: Pre- vs. Post-Fine-Tuning Comparison

The table below records the baseline validation-calibrated QDN metrics alongside placeholders for the completed 2-epoch hard-negative fine-tuning run (saved to `outputs/final/qdn_hard_neg/` and `hard_neg_summary.json`). Once the terminal run finishes, provide the output logs to populate the final post-fine-tuning columns.

Pathology Class	Baseline Test F1 ($\tau_{gate} = 0.30$)	Fine-Tuned Val F1 & Threshold	Fine-Tuned Test F1 ($\tau_{gate} = 0.30$)	Test TP / FP Count	Fine-Tuned Test AUPRC
Effusion	0.5312	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*
Pneumothorax	0.4695	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*
Cardiomegaly	0.4187	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*	*[Awaiting Run Output]*

8. Production-Ready Scripts for Repository Integration

This section provides the complete, verified scripts required to reproduce the Stage-1 gate evaluation, false-positive forensics, and hard-negative suppression fine-tuning from a clean checkout of the repository.

8.1 Post-Processing & Stage-1 Gate Evaluation (`evaluate_optimized.py`)

Sweeps decision thresholds exclusively on the validation split while applying the Stage-1 abnormality gate ($\tau_{gate} = 0.30$) and evaluates natural-prevalence F1 and AUPRC on the held-out test split.

```
import os
import numpy as np
from sklearn.metrics import f1_score, average_precision_score, roc_auc_score

FOCUS_CLASSES = ["Effusion", "Pneumothorax", "Cardiomegaly"]
GATE_THRESHOLD = 0.30

def evaluate_model_dir(base_dir="outputs/final/qdn", gate_thresh=GATE_THRESHOLD):
    print(f"=== Evaluating {base_dir} (Stage-1 Gate Threshold = {gate_thresh})
===")
    val_gate = np.load(os.path.join(base_dir, "gate", "val_probs.npy"))
    test_gate = np.load(os.path.join(base_dir, "gate", "test_probs.npy"))

    for cls_name in FOCUS_CLASSES:
        cls_dir = os.path.join(base_dir, f"cls_{cls_name}")
        val_p = np.load(os.path.join(cls_dir, "val_probs.npy"))
        val_y = np.load(os.path.join(cls_dir, "val_labels.npy"))
        test_p = np.load(os.path.join(cls_dir, "test_probs.npy"))
        test_y = np.load(os.path.join(cls_dir, "test_labels.npy"))

        # Apply Stage-1 Gate override on validation predictions
        val_p_gated = np.where(val_gate < gate_thresh, 0.0, val_p)

        best_th, best_val_f1 = 0.5, 0.0
        for th in np.linspace(0.05, 0.95, 91):
            f1 = f1_score(val_y, (val_p_gated >= th).astype(int), zero_division=0)
            if f1 > best_val_f1:
                best_val_f1, best_th = f1, th

        # Evaluate on held-out test split using validation-calibrated threshold
        test_p_gated = np.where(test_gate < gate_thresh, 0.0, test_p)
        test_preds = (test_p_gated >= best_th).astype(int)

        test_f1 = f1_score(test_y, test_preds, zero_division=0)
        test_auprc = average_precision_score(test_y, test_p)
        test_auroc = roc_auc_score(test_y, test_p)
        tp = int(((test_preds == 1) & (test_y == 1)).sum())
        fp = int(((test_preds == 1) & (test_y == 0)).sum())

        print(f"{cls_name:15s} | Test F1: {test_f1:.4f} (Val F1: {best_val_f1:.4f}
@ th={best_th:.2f}) "
              f"| TP: {tp:4d} | FP: {fp:4d} | AUPRC: {test_auprc:.4f} | AUROC:
{test_auroc:.4f}")

if __name__ == "__main__":
    evaluate_model_dir("outputs/final/qdn")
```

8.2 False-Positive Metadata Forensics (`inspect_top_fps.py`)

Identifies the highest-confidence false positives on the focus classes and cross-references them against radiograph view position (PA vs. AP) and co-existing multi-label abnormalities.

```
import os
import numpy as np
import pandas as pd

FOCUS_CLASSES = ["Effusion", "Pneumothorax", "Cardiomegaly"]

def inspect_false_positives(base_dir="outputs/final/qdn",
test_csv="splits/test.csv", top_k=10):
    df_test = pd.read_csv(test_csv)
    test_gate = np.load(os.path.join(base_dir, "gate", "test_probs.npy"))

    for cls_name in FOCUS_CLASSES:
        cls_dir = os.path.join(base_dir, f"cls_{cls_name}")
        test_p = np.load(os.path.join(cls_dir, "test_probs.npy"))
        test_y = np.load(os.path.join(cls_dir, "test_labels.npy"))

        # Isolate negative ground-truth instances sorted by predicted probability
        neg_indices = np.where(test_y == 0)[0]
        sorted_fp_idx = neg_indices[np.argsort(test_p[neg_indices])[::-1]][:top_k]

        print(f"\n=== Top {top_k} False Positives for {cls_name} ===")
        for idx in sorted_fp_idx:
            row = df_test.iloc[idx]
            img_id = row.get("Image Index", idx)
            view = row.get("View Position", "N/A")
            abn = int(row.get("Is_Abnormal", 0))
            findings = row.get("Finding Labels", "N/A")
            print(f"Image: {img_id} | Prob: {test_p[idx]:.4f} | Gate:
{test_gate[idx]:.4f} "
                  f"| View: {view} | Is_Abnormal: {abn} | True Findings:
{findings}")

if __name__ == "__main__":
    inspect_false_positives()
```

8.3 Native Hard-Negative Suppression Fine-Tuning (`finetune_qdn.py`)

The primary production fine-tuning script. Strictly loads `outputs/final/qdn/best_ema.pt` into the native `QueryDiscussNet` architecture, locks EMA warmup updates (`ema.updates = 5000`), oversamples multi-label confounder scans via `WeightedRandomSampler` , computes `HardNegativeSuppressionLoss` in FP32 outside `autocast` , and exports the updated checkpoints and evaluation arrays to `outputs/final/qdn_hard_neg/` .

```
import os, time, json, random
import yaml
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, WeightedRandomSampler
from sklearn.metrics import roc_auc_score, average_precision_score, f1_score

from data.patient_split import load_saved_splits, TARGET_11_CLASSES
from data.dataset import ChestXrayDataset
from models.query_discuss_net import QueryDiscussNet
from training.train_pipeline import ModelEMA, make_scheduler, predict_loader

COLS = TARGET_11_CLASSES + ["Is_Abnormal"]
FOCUS = ["Effusion", "Pneumothorax", "Cardiomegaly"]


class HardNegativeSuppressionLoss(nn.Module):
    """
    Computes BCE in float32 outside autocast (preventing NaN) while heavily
penalizing
    high-confidence false positives (y == 0) on focus classes and confounder
scans.
    """
    def __init__(self, focus_indices, fp_weight=3.5, gamma=2.0):
        super().__init__()
        self.focus_indices = focus_indices
        self.fp_weight = fp_weight
        self.gamma = gamma

    def forward(self, logits: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        logits = logits.float()
        y = y.float()
        bce = F.binary_cross_entropy_with_logits(logits, y, reduction="none")
        probs = torch.sigmoid(logits).detach()

        weights = torch.ones_like(bce)
        is_abn = y[:, -1:]  # (B, 1) -> 1 if scan has any abnormality

        for idx in self.focus_indices:
            neg_mask = (y[:, idx] == 0.0).float()
            conf_penalty = (probs[:, idx] ** self.gamma) * self.fp_weight * (1.0 +
0.5 * is_abn.squeeze(1))
            weights[:, idx] = 1.0 + neg_mask * conf_penalty

        return (bce * weights).mean()


def compute_metrics(p, y):
    out = {}
    for i, c in enumerate(COLS):
        out[c] = {
            "auroc": float(roc_auc_score(y[:, i], p[:, i])),
            "aupre": float(average_precision_score(y[:, i], p[:, i]))
```

8.4 Core QDN Architecture Reference (`models/query_discuss_net.py`)

Included for completeness so external collaborators have the exact model definition paired with `finetune_qdn.py`.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import timm

def sincos_2d(h: int, w: int, dim: int, device, dtype) -> torch.Tensor:
    assert dim % 4 == 0, "embed_dim must be divisible by 4"
    d = dim // 4
    omega = 1.0 / (10000 ** (torch.arange(d, device=device, dtype=torch.float32) /
d))
    ys, xs = torch.meshgrid(torch.arange(h, device=device, dtype=torch.float32),
                            torch.arange(w, device=device, dtype=torch.float32),
indexing="ij")
    oy = ys.flatten()[:, None] * omega[None]
    ox = xs.flatten()[:, None] * omega[None]
    return torch.cat([oy.sin(), oy.cos(), ox.sin(), ox.cos()], dim=1).to(dtype)

class ReadDiscussStep(nn.Module):
    def __init__(self, dim: int, heads: int, dropout: float, discuss: bool,
refine_tokens: bool):
        super().__init__()
        self.discuss, self.refine = discuss, refine_tokens
        self.n_q, self.n_kv = nn.LayerNorm(dim), nn.LayerNorm(dim)
        self.read = nn.MultiheadAttention(dim, heads, dropout=dropout,
batch_first=True)
        if discuss:
            self.n_s = nn.LayerNorm(dim)
            self.talk = nn.MultiheadAttention(dim, heads, dropout=dropout,
batch_first=True)
        self.n_f = nn.LayerNorm(dim)
        self.ffn = nn.Sequential(nn.Linear(dim, dim * 4), nn.GELU(),
nn.Dropout(dropout),
                                 nn.Linear(dim * 4, dim), nn.Dropout(dropout))
        if refine_tokens:
            self.n_t, self.n_qq = nn.LayerNorm(dim), nn.LayerNorm(dim)
            self.write = nn.MultiheadAttention(dim, heads, dropout=dropout,
batch_first=True)

    def forward(self, q: torch.Tensor, tokens: torch.Tensor):
        kv = self.n_kv(tokens)
        a, _ = self.read(self.n_q(q), kv, kv)
        q = q + a
        if self.discuss:
            s = self.n_s(q)
            t, _ = self.talk(s, s, s)
            q = q + t
        q = q + self.ffn(self.n_f(q))
        if self.refine:
            qq = self.n_qq(q)
            w, _ = self.write(self.n_t(tokens), qq, qq)
            tokens = tokens + w
        return q, tokens

class QueryDiscussNet(nn.Module):
    def __init__(self, num_classes: int = 11, backbone_name: str = "resnet50d"
```